

课程笔记

CS224N Lecture 3: Backpropagation and Neural Networks

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年 4 月 9 日

课程版本：2024 年中文讲义整理版

笔记来源：AI Course Notes Contributors

本版处理：移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权：笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture03>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言：课程概览	3
1.1 本章小结	3
2 神经网络基础回顾	3
2.1 从逻辑回归到神经网络	3
2.2 层的矩阵表示	3
2.3 NER 示例：命名实体识别	4
2.4 本章小结	4
3 激活函数：为什么需要非线性	4
3.1 从阈值函数到可微激活函数	4
3.2 常见激活函数一览	4
3.3 为什么非线性不可或缺	5
3.4 本章小结	5
4 梯度下降与梯度计算	5
4.1 随机梯度下降 (SGD) 回顾	5
4.2 本章小结	6
5 矩阵微积分：手动推导梯度	6
5.1 从单变量到多变量	6
5.2 梯度与 Jacobian 矩阵	6
5.3 链式法则的矩阵形式	7
5.4 逐元素激活函数的 Jacobian	7
5.5 线性层的 Jacobian	7
5.6 NER 网络的完整梯度推导	8
5.7 外积梯度的直觉	9
5.8 本章小结	9
6 反向传播算法	9
6.1 反向传播的本质	9
6.2 计算图	9
6.3 单节点的梯度传播规则	9
6.4 实例： $f(x, y, z) = (x + y) \cdot \max(y, z)$	10
6.5 分支处的梯度求和	10
6.6 三种基本操作的梯度直觉	10
6.7 高效反向传播：避免重复计算	11
6.8 通用反向传播算法	11
6.9 本章小结	11
7 自动微分与框架实现	11
7.1 自动微分	11
7.2 Forward/Backward API	12

7.3	本章小结	12
8	梯度检验：数值验证	12
8.1	数值梯度	12
8.2	为什么不直接用数值梯度训练？	13
8.3	本章小结	13
9	总结与延伸	13
9.1	讲者的核心总结	13
9.2	全课知识图谱	14
9.3	关键 Takeaways	14
9.4	拓展阅读	14

1 引言：课程概览

本节课是 Stanford CS224N (Natural Language Processing with Deep Learning) 的第三讲，由 Chris Manning 教授讲授。课程的核心目标是让学生理解神经网络训练的数学原理——如何**手动推导梯度**（矩阵微积分）以及如何**用算法自动计算梯度**（反向传播算法）。

课程分为三个部分：

本讲核心学习目标

1. 理解神经网络中梯度的数学推导——矩阵微积分 (Matrix Calculus)
2. 掌握反向传播算法 (Backpropagation) ——链式法则的高效实现
3. 建立“前向传播计算函数值、反向传播计算梯度”的直觉

1.1 本章小结

本讲聚焦于神经网络训练的数学基础。Manning 教授强调，虽然现代深度学习框架（如 PyTorch）已经自动化了梯度计算，但理解底层原理对于调试、设计新模型和理解训练行为至关重要。

2 神经网络基础回顾

2.1 从逻辑回归到神经网络

上一讲介绍了神经网络的基本概念：每个神经元本质上是一个小型的逻辑回归单元。与传统机器学习单个逻辑回归不同的是，神经网络将这些单元**级联**起来，形成多层结构。

神经网络的核心优势——表示学习

神经网络最强大的特性在于**中间层的自组织** (self-organization of intermediate representations)。网络不需要人工设计特征，而是自动学习“什么样的中间表示对最终任务有用”。这正是神经网络在大多数场景下优于传统机器学习的根本原因。

2.2 层的矩阵表示

当神经网络采用规则的层状结构时，每一层的计算可以用矩阵运算表示：

具体来说，一层神经网络的计算包含两步：

第一步：线性变换

$$z = Wx + b$$

其中：

- $x \in \mathbb{R}^m$ ：输入向量
- $W \in \mathbb{R}^{n \times m}$ ：权重矩阵
- $b \in \mathbb{R}^n$ ：偏置向量
- $z \in \mathbb{R}^n$ ：线性变换输出

第二步：非线性激活

$$a = f(z)$$

激活函数 f 逐元素应用： $f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)]$ 。

2.3 NER 示例：命名实体识别

Manning 教授使用一个具体的 NER（命名实体识别）任务来贯穿整个讲解：判断一个上下文窗口中心的词是否为地点名称。

该网络的完整计算过程：

1. 将 5 个词的词向量拼接为输入 $x \in \mathbb{R}^{5d}$
2. 经过隐藏层： $h = f(Wx + b)$
3. 点积得到分数： $s = u^T h$
4. 通过 sigmoid 得到概率： $J_t(\theta) = \sigma(s) = \frac{1}{1+e^{-s}}$

为什么用词嵌入而不是一-hot ？

One-hot 向量维度极高（词表大小）且稀疏，无法表达词与词之间的语义关系。词嵌入（word embeddings）将每个词映射为一个低维稠密向量，使得语义相近的词在向量空间中距离也近。在深度学习中， θ 不仅包含权重矩阵等传统参数，**词向量本身也是需要学习的参数**。

2.4 本章小结

神经网络通过将多个“小型逻辑回归”级联，自动学习中间表示。每一层的计算可以简洁地表示为矩阵乘法加偏置再加非线性激活： $a = f(Wx + b)$ 。接下来的问题是：如何训练这个网络？答案是梯度下降，而这需要我们计算梯度。

3 激活函数：为什么需要非线性

3.1 从阈值函数到可微激活函数

神经元模型的历史始于 McCulloch & Pitts（1943）的阈值单元：

阈值函数的问题在于它是分段常数函数——斜率处处为零（除了不连续点），因此**无法提供梯度信息**来指导学习。梯度学习的核心思想是“找到哪里更陡峭，朝那个方向走”——如同滑雪，你需要坡度来决定方向。

3.2 常见激活函数一览

Logistic (Sigmoid) 函数：

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

输出范围 $(0, 1)$ ，常用于输出层（映射到概率）。缺点：输出始终为正，且在两端梯度趋近于零（梯度消失）。

Tanh 函数：

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

输出范围 $(-1, 1)$ ，是 sigmoid 的缩放平移版本： $\tanh(z) = 2\sigma(2z) - 1$ 。

ReLU (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(z, 0)$$

正半轴斜率恒为 1，计算极快。虽然负半轴“死区”（梯度为零）看似矛盾，但实践中效果极好——它提供了一种自然的稀疏激活和特化机制。

ReLU 的“死区”问题

当 ReLU 神经元的输入持续为负时，梯度为零，该神经元“永久死亡”，无法再被激活。这就是所谓的 **dying ReLU** 问题。虽然在正常训练中这通常不是致命问题（网络中总有一部分神经元是活跃的），但在学习率过大或初始化不当时可能导致大量神经元死亡。

Leaky ReLU / Parametric ReLU: 在负半轴给予一个小斜率（如 0.01），避免完全死亡。Parametric ReLU 将负半轴斜率作为可学习参数。

Swish 和 GELU: 近年来在 Transformer 模型中广泛使用。

$$\text{Swish}(x) = x \cdot \sigma(x), \quad \text{GELU}(x) = x \cdot P(X \leq x), \quad X \sim \mathcal{N}(0, 1)$$

两者形状类似：正半轴近似 $y = x$ ，负半轴有一小段“弯曲”区域提供梯度。

3.3 为什么非线性不可或缺

线性层的叠加

如果只做矩阵乘法而没有非线性：

$$h_2 = W_2(W_1x) = (W_2W_1)x = W'x$$

多层网络在表示能力上与单层完全等价！非线性激活函数是使神经网络成为**万能函数逼近器**（universal function approximator）的关键。

线性网络在学习理论中的价值

Manning 教授指出了—个有趣的现象：虽然线性神经网络在**表示能力**上没有优势（等价于单层线性变换），但在**学习动力学**上却有独特性质。理论界有相当多论文研究线性网络的学习行为，因为多层结构即使没有非线性也会影响优化路径。

3.4 本章小结

激活函数是神经网络能够学习复杂非线性映射的关键。从历史上的阈值函数到 sigmoid/tanh，再到 ReLU 及其变体，直到 Transformer 时代的 Swish/GELU，激活函数的演进反映了对“梯度流通性”和“计算效率”的不断追求。核心原则是：保留足够的梯度信号使学习成为可能。

4 梯度下降与梯度计算

4.1 随机梯度下降 (SGD) 回顾

神经网络的训练依赖于**随机梯度下降** (Stochastic Gradient Descent)：

$$\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla_{\theta} J(\theta)$$

其中：

- θ ：模型的所有参数（包括权重矩阵 W 、偏置 b 、词向量等）
- α ：学习率（step size）
- $\nabla_{\theta} J(\theta)$ ：损失函数关于参数的梯度

核心问题：如何计算 $\nabla_{\theta} J(\theta)$ ？有两种方法：

1. **手动推导**（By hand）——矩阵微积分
2. **算法自动计算**（Algorithmically）——反向传播算法

本讲两种方法都会讲解。

4.2 本章小结

SGD 是训练神经网络的核心算法，其关键步骤是计算损失函数关于所有参数的梯度。在深度学习中，参数 θ 不仅包括传统的权重和偏置，还包括词嵌入等数据表示参数。

5 矩阵微积分：手动推导梯度

5.1 从单变量到多变量

Manning 教授给出了一个关键的思维口诀：

矩阵微积分的核心口诀

“Multivariable calculus is just like single-variable calculus if you use matrices.”
（多变量微积分就是单变量微积分——只不过用矩阵代替标量。）

回顾单变量微积分的基本例子： $f(x) = x^3$ ，导数 $f'(x) = 3x^2$ 。导数衡量的是斜率——输入的小变化如何被放大到输出上。例如在 $x = 4$ 处， $f'(4) = 48$ ，意味着 x 从 4 变到 4.01 时， $f(x)$ 大约增加 $48 \times 0.01 = 0.48$ （即从 64 变为约 64.48）。

5.2 梯度与 Jacobian 矩阵

当函数有 n 个输入和 1 个输出时（ $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ），梯度是一个向量：

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

当函数有 n 个输入和 m 个输出时（ $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ），偏导数构成一个 $m \times n$ 的矩阵，称为 **Jacobian 矩阵**：

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}$$

Jacobian 在神经网络中的角色

神经网络的每一层都是一个多输入多输出的函数。例如一个隐藏层 $h = f(Wx + b)$ 将 m 维输入映射到 n 维输出。该层的 Jacobian 就是一个 $n \times m$ 的矩阵，描述了每个输出维度对每个输入维度的敏感度。

5.3 链式法则的矩阵形式

对于复合函数，链式法则变为 Jacobian 矩阵的乘法。如果 $h = f(g(x))$ ，则：

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial g} \cdot \frac{\partial g}{\partial x}$$

这与单变量的链式法则 $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$ 形式完全一致——只是标量乘法变成了矩阵乘法。

5.4 逐元素激活函数的 Jacobian

对于逐元素激活函数 $h = f(z)$ （其中 $h_i = f(z_i)$ ），其 Jacobian 是一个**对角矩阵**：

$$\left(\frac{\partial \mathbf{h}}{\partial \mathbf{z}} \right)_{ij} = \begin{cases} f'(z_i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

即：

$$\frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \text{diag}(f'(\mathbf{z}))$$

对角结构的直觉

为什么是对角矩阵？因为激活函数是**逐元素**应用的： h_i 只依赖于 z_i ，不依赖于 z_j ($j \neq i$)。因此非对角元素 ($i \neq j$) 的偏导数为零。

5.5 线性层的 Jacobian

对于线性变换 $z = Wx + b$ ：

$$\frac{\partial z}{\partial x} = W, \quad \frac{\partial z}{\partial b} = I$$

类比单变量： $y = wx + b$ 的导数关于 x 是 w ，关于 b 是 1。矩阵版本中， w 变为 W ，1 变为单位矩阵 I 。

对于点积 $s = u^T h$ ：

$$\frac{\partial s}{\partial h} = u^T$$

5.6 NER 网络的完整梯度推导

现在将所有 Jacobian 组合起来，对 NER 示例网络推导梯度。网络结构为：

$$s = u^T h, \quad h = f(z), \quad z = Wx + b$$

计算 $\frac{\partial s}{\partial b}$ ：

$$\frac{\partial s}{\partial b} = \frac{\partial s}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial b} = u^T \cdot \text{diag}(f'(z)) \cdot I$$

化简后：

$$\frac{\partial s}{\partial b} = u^T \circ f'(z)$$

其中 $\circ$ 表示 **Hadamard 乘积**（逐元素乘法）。

Hadamard 乘积

Hadamard 乘积（记作 $\circ$ 或 $\odot$ ）是将两个相同维度的向量/矩阵**逐元素相乘**。不同于点积（结果是标量），Hadamard 乘积的结果维度与输入相同。在神经网络的梯度计算中，它出现在对角 Jacobian 乘以向量时——等价于两个向量的逐元素乘法。

上游梯度 δ 的概念：

注意到计算 $\frac{\partial s}{\partial b}$ 和 $\frac{\partial s}{\partial W}$ 时，前两项是共享的：

$$\delta = \frac{\partial s}{\partial h} \cdot \frac{\partial h}{\partial z} = u^T \circ f'(z)$$

δ 称为**上游梯度**（upstream gradient）或**误差信号**（error signal）。它可以计算一次、复用多次，这是反向传播算法效率的关键。

于是：

- $\frac{\partial s}{\partial b} = \delta$ （因为 $\frac{\partial z}{\partial b} = I$ ）
- $\frac{\partial s}{\partial W} = \delta^T x^T$ （外积形式）

计算 $\frac{\partial s}{\partial W}$ ：

按照严格的 Jacobian 定义， $\frac{\partial s}{\partial W}$ 应该是一个 $1 \times nm$ 的行向量（因为 s 是标量， W 有 nm 个参数）。但在实际工程中，我们采用**形状约定**（shape convention），将梯度整形为与参数**相同的形状**，以便直接做 SGD 更新 $W^{\text{new}} = W^{\text{old}} - \alpha \frac{\partial s}{\partial W}$ 。

最终结果：

$$\frac{\partial s}{\partial W} = \delta^T x^T \in \mathbb{R}^{n \times m}$$

这是 δ^T ($n \times 1$) 与 x^T ($1 \times m$) 的**外积**。

Jacobian 形式 vs 形状约定

数学上，Jacobian 矩阵有严格的定义和形状，链式法则在 Jacobian 形式下是矩阵乘法。但在工程实现中，为了方便 SGD 更新，我们通常将梯度“reshape”为与参数相同的形状。这两种约定混用可能导致困惑——Manning 教授建议：可以先用 Jacobian 做正确的数学推导，最后再 reshape 结果；或者始终按形状约定做，但要灵活使用转置。

5.7 外积梯度的直觉

为什么 $\frac{\partial s}{\partial W} = \delta^T x^T$ 是对的？考虑单个权重 W_{ij} ：它只参与计算 $z_i = \sum_k W_{ik} x_k + b_i$ ，即只连接输入 x_j 到隐藏层 z_i 。因此：

$$\frac{\partial s}{\partial W_{ij}} = \delta_i \cdot x_j$$

这恰好是外积 $\delta^T x^T$ 的第 (i, j) 个元素。

5.8 本章小结

矩阵微积分的核心是：用 Jacobian 矩阵表示多输入多输出函数的导数，用矩阵乘法表示链式法则。对于神经网络的常见操作（线性层、激活函数、点积），Jacobian 都有简洁的封闭形式。关键概念是**上游梯度** δ ——它在不同参数的梯度计算中被复用，避免重复计算。

6 反向传播算法

6.1 反向传播的本质

反向传播算法

1. **使用链式法则**：将复合函数的梯度分解为局部梯度的乘积
2. **存储中间结果**：避免重复计算——同一个上游梯度被多个下游节点共享

这就是反向传播算法的全部。

6.2 计算图

为了系统化地实现反向传播，我们将计算过程表示为**计算图**（Computation Graph）：
计算图中：

- **源节点**：输入变量 (x, W, b, u)
- **内部节点**：运算操作（矩阵乘法、加法、激活函数、点积）
- **边**：传递操作的结果

前向传播（Forward Propagation）：沿计算图从左到右，依次计算每个节点的值。

反向传播（Backward Propagation / Backpropagation）：沿计算图从右到左，依次计算梯度。

6.3 单节点的梯度传播规则

对于计算图中的任意一个节点，设其计算为 $h = f(z)$ ：

$$\underbrace{\frac{\partial s}{\partial z}}_{\text{下游梯度}} = \underbrace{\frac{\partial s}{\partial h}}_{\text{上游梯度}} \times \underbrace{\frac{\partial h}{\partial z}}_{\text{局部梯度}}$$

这就是链式法则在计算图中的体现。当节点有多个输入时，对每个输入分别计算局部梯度，然后分别与上游梯度相乘。

6.4 实例： $f(x, y, z) = (x + y) \cdot \max(y, z)$

Manning 教授用一个具体的非神经网络例子来演示反向传播的过程。

前向传播：

- $a = x + y = 1 + 2 = 3$
- $b = \max(y, z) = \max(2, 0) = 2$
- $f = a \cdot b = 3 \times 2 = 6$

局部梯度：

- 加法节点： $\frac{\partial a}{\partial x} = 1, \frac{\partial a}{\partial y} = 1$
- max 节点： $\frac{\partial b}{\partial y} = 1 \ (y > z), \frac{\partial b}{\partial z} = 0$
- 乘法节点： $\frac{\partial f}{\partial a} = b = 2, \frac{\partial f}{\partial b} = a = 3$

反向传播（从右到左）：

1. $\frac{\partial f}{\partial f} = 1$
2. 乘法节点： $\frac{\partial f}{\partial a} = b = 2, \frac{\partial f}{\partial b} = a = 3$
3. 加法节点： $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot 1 = 2$, 从加法到 y : $\frac{\partial f}{\partial a} \cdot 1 = 2$
4. max 节点： 到 y : $\frac{\partial f}{\partial b} \cdot 1 = 3$, 到 z : $\frac{\partial f}{\partial b} \cdot 0 = 0$
5. y 有两条路径： $\frac{\partial f}{\partial y} = 2 + 3 = 5$

最终结果： $\frac{\partial f}{\partial x} = 2, \frac{\partial f}{\partial y} = 5, \frac{\partial f}{\partial z} = 0$ 。

验证： 将 y 从 2 改为 2.1, 则 $a = 3.1, b = 2.1, f = 3.1 \times 2.1 = 6.51$, 变化量 $\approx 0.51 \approx 5 \times 0.1$, 与梯度 5 吻合。

6.5 分支处的梯度求和

上例中 y 同时参与了加法和 max 两个计算，因此它的总梯度是两条路径上梯度的**和**：

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial y} + \frac{\partial f}{\partial b} \cdot \frac{\partial b}{\partial y} = 2 \times 1 + 3 \times 1 = 5$$

这就是多元链式法则的体现。

6.6 三种基本操作的梯度直觉

三种基本操作的梯度行为

- **加法 (+)：** 分发梯度——上游梯度原封不动地传给两个输入
- **Max：** 路由梯度——上游梯度全部传给较大的那个输入，另一个输入得到零
- **乘法 (×)：** 切换梯度——上游梯度乘以另一个输入的值传给当前输入

Max 操作导致梯度消失

Max 操作对“输的那一方”梯度为零——这与 ReLU 的行为一致 ($\text{ReLU}(x) = \max(0, x)$)。当 $x < 0$ 时，ReLU 的梯度为零，意味着该方向的信息完全被阻断。这从计算图的角度解释了 dying ReLU 现象。

6.7 高效反向传播：避免重复计算

如果分别独立地计算 $\frac{\partial s}{\partial b}$ 、 $\frac{\partial s}{\partial W}$ 、 $\frac{\partial s}{\partial x}$ 、 $\frac{\partial s}{\partial u}$ ，会发现它们共享大量的中间计算（上游梯度 δ ）。反向传播的核心效率在于：**按照拓扑排序的逆序一次性遍历计算图**，每个中间梯度只计算一次。

反向传播的计算复杂度

如果正确实现，反向传播的时间复杂度与前向传播的时间复杂度**同阶**（Big-O 相同）。如果你的反向传播比前向传播慢很多，说明你在某处做了重复计算。

6.8 通用反向传播算法

算法流程：

前向传播：

1. 对计算图做**拓扑排序**，确保每个节点只依赖已计算的节点
2. 按拓扑序遍历，对每个节点调用其 `forward` 方法计算输出值

反向传播：

1. 初始化输出梯度： $\frac{\partial s}{\partial s} = 1$
2. 按**逆拓扑序**遍历节点
3. 对每个节点：下游梯度 = 上游梯度 $\times$ 局部梯度
4. 对分支节点：将多条路径的梯度**求和**

该算法适用于任意有向无环图（DAG），不要求网络是规整的层状结构。

6.9 本章小结

反向传播算法是链式法则在计算图上的高效实现。其核心是“一次前向、一次反向”，利用中间结果共享避免重复计算。对于任意 DAG 结构的计算图，反向传播的复杂度与前向传播同阶。

7 自动微分与框架实现

7.1 自动微分

既然反向传播算法是如此系统化，能否让计算机**完全自动**完成？

自动微分的历史

早期的深度学习框架 **Theano**（蒙特利尔大学开发）尝试了完全符号化的自动微分：给定前向计算的符号表达式，自动推导出反向传播的符号表达式。但这种方式过于重量级，难以灵活处理各种情况。

现代深度学习框架（PyTorch、TensorFlow 等）采用了一种折中方案：

- **框架负责**：管理计算图、执行拓扑排序、运行前向/反向遍历、传递上游梯度
- **用户负责**：为每种操作实现 `forward`（前向计算）和 `backward`（局部梯度计算）方法

7.2 Forward/Backward API

```
1 class MultiplyGate(object):
2     def forward(self, x, y):
3         z = x * y
4         self.x = x # 缓存输入, 供 backward 使用
5         self.y = y
6         return z
7
8     def backward(self, dz):
9         dx = dz * self.y # 上游梯度 * 对方输入
10        dy = dz * self.x
11        return [dx, dy]
```

Listing 1: 乘法门的 Forward/Backward 实现

backward 依赖 forward 的缓存

注意一个关键细节：`backward` 方法需要用到前向传播时的输入值（如乘法门中的 x 和 y ）。因此 `forward` 方法必须将这些值缓存起来（存为类的属性）。这就是为什么在 PyTorch 的 `autograd.Function` 中，`forward` 需要用 `ctx.save_for_backward()` 保存张量。

实际上，PyTorch 已经预定义了大量常用操作的 `forward` 和 `backward` 实现。用户只需像搭积木一样组合这些操作，PyTorch 会自动管理计算图和梯度传播。这正是“高中生也能做深度学习项目”的原因。

7.3 本章小结

现代深度学习框架将反向传播的“基础设施”（计算图管理、梯度传播）自动化，用户只需为每个操作定义前向计算和局部梯度。常见操作（线性层、激活函数、损失函数等）都已预实现，使用者通常不需要手动编写 `backward` 方法。但理解底层原理对于调试和设计新操作至关重要。

8 梯度检验：数值验证

8.1 数值梯度

如何验证你手动推导或实现的梯度是否正确？可以用**数值梯度**（Numerical Gradient）进行检验：

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x-h)}{2h}$$

其中 h 是一个很小的数（通常取 10^{-4} ）。

双侧差分 vs 单侧差分

一定要用**双侧差分** (centered difference) $\frac{f(x+h)-f(x-h)}{2h}$ ，而不是单侧差分 $\frac{f(x+h)-f(x)}{h}$ 。双侧差分的误差是 $O(h^2)$ ，而单侧差分的误差是 $O(h)$ ，精度差了一个数量级。

8.2 为什么不直接用数值梯度训练？

数值梯度计算简单，为什么不直接用它代替反向传播？因为它**极其缓慢**：需要对每一个参数单独做一次前向计算。如果模型有 N 个参数，就需要 $2N$ 次前向传播（双侧差分）。而反向传播只需要**一次前向 + 一次反向**就能得到所有参数的梯度。

梯度检验的正确用法

数值梯度仅用于**验证**你的 backward 实现是否正确，绝不用于训练。检验方法：

1. 用反向传播计算梯度 g_{analytic}
2. 用数值差分估计梯度 g_{numeric}
3. 检查 $|g_{\text{analytic}} - g_{\text{numeric}}| < \epsilon$ (通常 $\epsilon \sim 10^{-5}$)

在 PyTorch 已预实现大部分操作的今天，梯度检验的需求减少了，但在实现**自定义操作**时仍然是必不可少的验证手段。

8.3 本章小结

数值梯度提供了一种“暴力但可靠”的梯度估计方法，是验证解析梯度实现正确性的黄金标准。使用双侧差分可以获得更高的精度。但由于计算量与参数数量成正比，数值梯度只适合验证，不适合训练。

9 总结与延伸

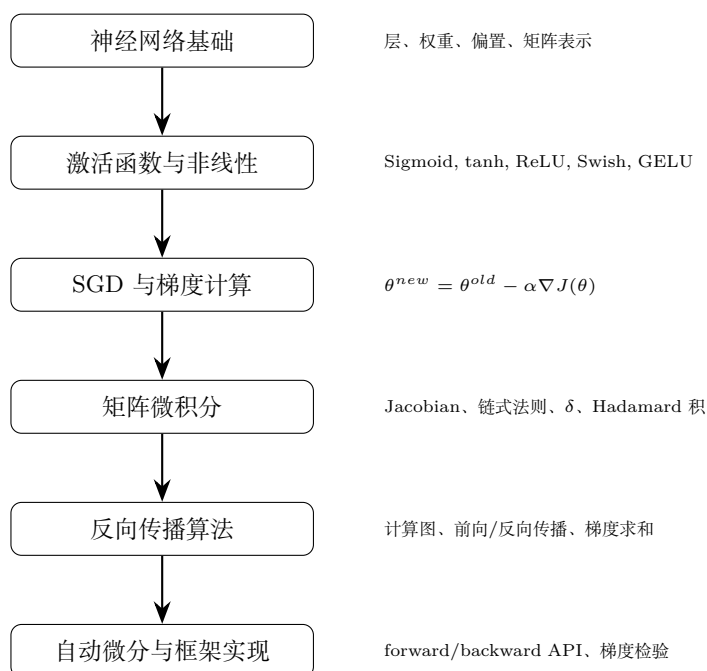
9.1 讲者的核心总结

Manning 教授在课程结尾总结了以下要点：

1. **反向传播 = 链式法则的高效实现**：前向传播计算函数值，反向传播计算梯度
2. **理解底层原理很重要**：虽然现代框架（PyTorch 等）自动化了一切，但理解反向传播的数学有助于调试和设计新架构
3. **梯度消失/爆炸**：后续讲解 RNN 时会看到，多层梯度连乘可能导致梯度指数级增长或衰减
4. **实践中的平衡**：框架预定义了常用操作的 forward/backward，用户像拼积木一样组合即可

⁰Slides 第 85 页（最后一页）。

9.2 全课知识图谱



9.3 关键 Takeaways

五条核心原则

1. **非线性是必须的**：没有激活函数，再多层也只是一个线性变换
2. **矩阵微积分 = 单变量微积分 + 矩阵**：Jacobian、链式法则在形式上与标量版完全类似
3. **δ （上游梯度）是核心概念**：它代表从损失函数回传到当前位置的“误差信号”，是多个参数梯度共享的部分
4. **反向传播的效率来自复用**：按逆拓扑序一次遍历，每个中间梯度只计算一次
5. **形状约定简化工程实现**：将梯度整形为与参数相同的形状，使 SGD 更新变为简单的减法

9.4 拓展阅读

- CS224N 官方讲义 (Lecture Notes) 与矩阵微积分教程: <https://web.stanford.edu/class/cs224n/>
- Stanford Math 51 在线教材 (线性代数与多变量微积分): <http://web.stanford.edu/class/math51/textbook.html>
- Karpathy, “Yes you should understand backprop”: <https://karpathy.medium.com/yes-you-should-understand-backprop-e2f06eab496b>
- Goodfellow et al., *Deep Learning* Chapter 6: Deep Feedforward Networks
- Justin Johnson, “Backpropagation for a Linear Layer”: <https://web.eecs.umich.edu/~justincj/teaching/eecs442/notes/linear-backprop.html>

- PyTorch 官方教程 – Autograd: https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html